Homework 7

Question 1 : Build a small FastAPI backend for a **Blog Management System** that demonstrates:

- JWT-based authentication
- Role-based authorization
- Linting with Ruff and pre-commit hooks

create **Blog Management System** with these roles:

- reader
- writer
- moderator

Functional Requirements

1. Authentication

Implement a login endpoint that accepts username and password and returns a JWT access token.

Required behavior:

- valid credentials → token returned
- invalid credentials → 401 Unauthorized

2. Authorization

Implement role-based access control using the roles above.

3. Routes

Your API must include at least these routes:

- POST /auth/login → login and get JWT token
- GET /posts → any authenticated user can view posts
- POST /posts → only writer and moderator can create posts

- DELETE /posts/{id} → only moderator can delete posts

Database Requirement (MySQL)

Use **MySQL** to store user details.

Table: users

Create a table with the following fields:

- id (primary key)
- username (unique)
- password (store as **hashed password**)
- role (reader, writer, moderator)

Requirements

- Do NOT store plain text passwords — store only **hashed passwords**
- During login, fetch user from DB and verify password using hashing
- Generate JWT token only after successful verification

Expected Behavior to Demonstrate

Your implementation must clearly show these cases:

- 200 OK → successful authenticated request
- 201 Created → successful post creation
- 401 Unauthorized → no token / invalid login / invalid token
- 403 Forbidden → valid token but insufficient role

Linting Requirement

Set up:

- ruff
- pre-commit

You must demonstrate:

- a linting error before commit
- pre-commit hook running during commit
- at least 1 or 2 issues fixed
- successful commit after resolving issues

Submission Requirements

Screenshots

Include screenshots of the following:

Authentication

- successful login returning JWT token
- failed login returning 401 Unauthorized

Authorization

- GET /users/me without token returning 401 Unauthorized
- POST /posts using reader role returning 403 Forbidden
- POST /posts using writer or moderator returning 201 Created
- DELETE /posts/{id} using writer returning 403 Forbidden
- DELETE /posts/{id} using moderator returning success
- GET /posts using valid token returning 200 OK

Linting / Pre-commit

- terminal screenshot showing linting error before commit
- terminal screenshot showing 1 or 2 issues fixed
- terminal screenshot showing successful commit

Single-Agent ReAct + MRKL Bike-Share Pass Optimizer (Public Data)

Build a single agent that recommends whether a rider should buy a monthly bike-share membership or pay per ride/minute, using a ReAct loop for control and MRKL tools for capability. The agent must read a period of public trip data and the system's official pricing/policy page , then output a cited recommendation with a transparent **Thought +Action + Observation + Final Answer** trace.

Choose ONE city (both data & policy are public)

- **New York City — Citi Bike**
  - Trip data (CSV, monthly): Citi Bike System Data. (Citi Bike NYC)
  - Pricing/policy: Citi Bike Pricing page. (Citi Bike NYC)

- **Chicago — Divvy**
  - Trip data: Divvy System Data / Divvy Trips Dashboard. (divvybikes.com)
  - Pricing/policy: Divvy Pricing page (note that prices change; cite page + date). (divvybikes.com)

- **San Francisco Bay Area — Bay Wheels**
  - Trip data: Bay Wheels System Data. (Lyft)
  - Pricing/policy: Bay Wheels pricing page on the same site (follow "Pricing" link from system page).

- **Washington, DC — Capital Bikeshare**
  - Trip data: Capital Bikeshare System Data. (capitalbikeshare.com)
  - Pricing/policy: Capital Bikeshare Pricing page (recent changes possible—include date). (capitalbikeshare.com)

Deliverables

- A small web UI that:

    - Accepts: a **month** CSV from the chosen city (user uploads one file) and a **URL** to the official pricing page.
    - Shows a **timeline** of agent thinking process or delegation of tasks to tools.
    - Shows a **cost comparison**: pay-per-ride/minute vs monthly membership, with any **overage/minute surcharges** and **included ride durations** applied per policy.
    - Includes **citations** to the pricing/policy dataset sections you used.

- A **single agent** using a **ReAct loop** + **MRKL tools** .
- Per-step logs: tool, args hash, latency, success/failure, stop reason.

Tools (MRKL) implement as pure functions with JSON schemas

1. csv_sql
   input: { "sql": string }
   output: { "rows": Array<object>, "row_count": number, "source": "uploaded.csv" }
   Behavior: run read-only SQL over the uploaded trips.

2. policy_retriever
   input: { "url": string, "query": string, "k"?: number }
   output: { "passages": [{ "text": string, "source": string, "score": number }] }
   Behavior: fetch the pricing page URL, extract text, and return **short, quotable** snippets (membership price, included ride minutes, per-minute ebike surcharges, unlock fees, overage rules).

3. calculator
   input: { "expression": string, "units"?: string } (whitelist ^[0-9+\\-*/().\\s]+$)
   output: { "value": number, "units"?: string }
   Behavior: safe arithmetic (no eval).

Common pattern: every tool must returns { "success": bool, "data"?: any, "error"?: string, "source"?: string, "ts"?: string }.

What the agent must produce

1. **Decision**: “Buy Monthly Membership” or “Pay Per Ride/Minute”.

2. **Justification** (3–6 sentences) with **citations** to pricing policy snippets (e.g., included minutes, ebike surcharge, unlock fees, membership price).

3. **Cost breakdown for the uploaded month**:

    - Total cost under **pay-per-use** (apply policy: included minutes per ride, minute-based surcharges, unlock fees; treat missing fields conservatively).

    - Monthly membership cost and any residual per-minute charges or unlock fees that still apply to members.

    - **Break-even** rides/time vs actual.

    - A small **table** by week: ride count, average duration, ebike usage share (if available in the city’s schema), spend.

4. **Assumptions & caveats** (e.g., if ebike/classic indicators are missing, assume classic; or infer from columns the dataset provides).

UI Requirements

- One file upload (CSV), one text field (pricing URL), and a **Run** button.

- A **timeline** panel showing Thought/Action/Observation/Final Answer.

- A **results** panel with the cost table, final decision, and citations (include the pricing page URL + capture date/time).

- A metrics strip: **steps**, **total time**, **stop reason**.

Acceptance Cases

Run your agent on **two different months or two station pairs** and include:

1. A case where **membership wins** (many, longer rides and/or frequent ebike use).
2. A case where **pay-per-use wins** (few/short rides).

For each scene:

- One paragraph explaining the decision with numbers.
- The **Stop Reason** and total steps.
- Start with NYC (Citi Bike) or Chicago (Divvy) if you want the largest, well-documented datasets. (Citi Bike NYC) (use a small subset)
- Policy pages can change—**save the text** you cite and include the date. (Citi Bike NYC)
- If a month's CSV is very large, filter early (station pair + commute windows) before computing costs. (my recommendation is to use a subset of the dataset, minimum 500 rows and keep your queries related to those rows)

References

- Citi Bike System Data / Pricing. (Citi Bike NYC)
- Divvy System Data / Pricing. (divvybikes.com)
- Bay Wheels System Data. (Lyft)
- Capital Bikeshare System Data / Pricing. (capitalbikeshare.com)